# I/O Profiling with Darshan

## NHR I/O Training

Julius Athenstaedt

NHR-Südwest

November 2026




**Johannes Gutenberg-Universität Mainz**
NHR-Südwest | I/O Training Workshop



# Agenda

- **Theory**
  - Why parallel I/O is hard to measure
  - What is Darshan
  - How Darshan intercepts I/O calls
  - What Darshan records
  - From per-process data to one log
  - What can I learn with Darshan?
- **Hands-on**
  - Build Darshan
  - HACC-IO on different file systems
  - Find the bottleneck
- **Results**
  - Generating a report
  - What we learned with Darshan

# The parallel I/O stack

- Application code
- High-level I/O library (HDF5, PnetCDF, ADIOS, ...)
- MPI-IO
- Parallel filesystem (Lustre, GPFS, ...)
- Physical storage (disk, SSD, burst buffer, ...)

A single `write()` call from the application's point of view can fan out into many operations across many storage systems, each can silently reshape the access pattern.

# Why is I/O performance hard to reason about?

- Distributed across many processes and storage servers
- Layers interact: an efficient-looking application pattern can be pathological once it hits the filesystem (many small, non-contiguous accesses).
- Shared, contended resource $\rightarrow$ other jobs on the same filesystem affect your measured performance.
- Most applications don't self-report I/O behavior

# What Darshan does

- Lightweight, scalable I/O **characterization** tool from Argonne National Laboratory
- Low overhead allows *always on*
- Answers *how* an application uses I/O subsystems: which files, how much data, which interfaces, how much time.
- Complements (doesn't replace) full per-operation tracing like DFTracer

# From job to report

1. **Intercept** the application's I/O calls
2. **Record** statistics per process, in memory
3. **Merge** all processes into one `.darshan` log at job shutdown
4. **Analyze** the log

## darshan-runtime: steps 1–3

Library that runs inside your job

## darshan-util: step 4

Tools to read and evaluate `.darshan` logs (incl. PyDarshan)

# Darshans design principle

*“Modules should not perform any I/O or communication within wrapper functions.”*

- Darshan records data **independently per process** while the application runs $\rightarrow$ no communication, no I/O of its own, during the job.
- All per-process data is merged into one log only at **shutdown**.
- $\rightarrow$ A profiler that does I/O while measuring I/O would distort exactly what it’s trying to observe.

# When is “shutdown”?

## MPI applications

Darshan starts in `MPI_Init` and writes **one log per job** in `MPI_Finalize`.

## Non-MPI applications (since 3.2.0)

Requires `DARSHAN_ENABLE_NONMPI=1` and `LD_PRELOAD`. The log is written when the process exits → **one log per process**, e.g. Python with worker processes produces several logs.

### No shutdown, no log

A crashed or killed process never reaches this point.

# Step 1: intercepting I/O calls

Darshan places **wrapper functions** in front of the I/O functions your application calls. Two ways to install them:

## 1. Build-time instrumentation

Darshan is linked into the executable, so its wrappers are called instead of the original I/O functions. Requires a rebuild.

## 2. Runtime instrumentation via LD_PRELOAD

Preload Darshan's shared library at process start. Works on already-compiled, dynamically-linked executables, including Python and other interpreted languages.

## Statically linked executables

`LD_PRELOAD` **does not work for statically linked executables**. Those must be instrumented at build time.

# LD_PRELOAD: what the dynamic linker does

- Every dynamically linked ELF executable is started by the dynamic linker (`ld.so`), which loads all shared libraries before `main()` runs.
- `LD_PRELOAD` = list of ELF shared objects "to be loaded before all others" (`man ld.so`).
  - Separated by colons or spaces, added to the link map left to right.
  - Three sources, in this order: `LD_PRELOAD` env var, `ld.so --preload` (glibc $\geq$ 2.30), `/etc/ld.so.preload`.
- Resulting symbol lookup order (observed with `LD_DEBUG=symbols`): **executable** $\rightarrow$ **preloaded libs** $\rightarrow$ **DT_NEEDED libs (e.g. `libc.so.6`)**.
- Calls into shared libraries go through Procedure Linkage Table (PLT) and Global Offset Table (GOT)

# LD_PRELOAD: interposing a libc symbol

- Wrapper library defines a function with the **same name and signature** as the libc function, e.g. `int open(const char *path, int flags, ...)`.
- Wrapper gets the "real" function via `dlsym(RTLD_NEXT, "open")`:
  - `RTLD_NEXT` = next occurrence of the symbol in the search order *after* the current object, i.e. libc's open.
  - Also chains through multiple preloaded layers.
- The **first** object that defines a symbol wins ⇒ a preloaded `open()` *interposes* libc's `open()` for every caller that resolves it dynamically.

# LD_PRELOAD: what it cannot see

- **Statically linked executables**: no dynamic linker, so `LD_PRELOAD` is not evaluated at all.
- **libc-internal calls**: e.g. `fopen()` opens the file through libc's own internal path, not through the exported open symbol.
- **Code that bypasses libc**: raw `syscall` instructions (inline assembly, runtimes with their own syscall layer) or `syscall(SYS_openat, ...)`. Check `pmem/syscall_intercept` to overcome that.
- **Memory-mapped I/O**: loads/stores to an `mmap`ed region are not function calls.
- Only functions somebody wrote a wrapper for are seen. Every new libc variant (open64, openat, __open_2, ...) needs its own wrapper.
- `exec()` starts a new program on the same process and therefore resets all memory mapping $\rightarrow$ lose state

# Modules: interfaces vs. extra views

Each module either wraps an **I/O interface** or adds a different **view** on top of it.

| | Built + active by default | Built, enable at runtime | Opt-in at build time |
|---|---|---|---|
| I/O interfaces | POSIX, STDIO, MPI-IO | | HDF5, PnetCDF, DAOS/DFS |
| File system info | Lustre (if `lustreapi` found) | | |
| Extra views | HEATMAP | DXT | |
| Non-I/O | | | APMPI, APXC, LDMS, MDHIM |

DXT: `DARSHAN_MOD_ENABLE=DXT_POSIX,DXT_MPIIO`

(deprecated, still works: `DXT_ENABLE_IO_TRACE=1`)

# What the default modules count

| | Data operations | Metadata operations |
|---|---|---|
| POSIX | `read`/`write`, p*/*v variants, `aio_*` | open/`creat`/`mkstemp`, dup, `lseek`, `stat`, `fsync`, `rename`, `mmap` |
| MPI-IO | independent, collective, split-collective, non-blocking reads/writes | independent/collective opens, `set_view`, `sync` |
| STDIO | `fread`/`fwrite`, `fprintf`, `fscanf`, … | `fopen`/`fdopen`, `fseek`, `fflush` |
| Lustre | – | no operations: stripe size/count, OSTs, pool per file |

All three interfaces additionally record: bytes, access-size histogram, read/write/metadata time, fastest/slowest rank, variance across ranks.

# Over time: HEATMAP and DXT

Counters say *how much*, these two say *when*.

## HEATMAP (default)

Bytes read/written per time bin, per rank and per interface, summed over all files. Max. 200 bins starting at 0.1 s. Bin width doubles when the run outgrows them → long runs get coarse bins.

## DXT (opt-in)

Every single read/write: file, rank, offset, length, start/end time. POSIX and MPI-IO only, memory-limited (default 2 MiB per module).

# Blind spots

- **`mmap`**: only wrapped in statically linked executables. With `LD_PRELOAD` the counter is `-1`. Even when wrapped, only the call is counted, not the bytes moved through the mapping.
- **`stat` on glibc $\geq$ 2.33**: Darshan wraps only the old `__xstat` symbols, newer binaries call `stat()` directly $\rightarrow$ not counted.
- **Not wrapped at all**: `unlink`, `mkdir`, `readdir`, `chmod`, …
- **Excluded paths**: files under `/etc`, `/usr`, `/lib`, `/opt`, `/var`, `/proc`, `/sys`, … are never recorded.

## Consequence

Metadata-heavy workloads look lighter in Darshan than they really are.

# Shared-file reduction, at job shutdown

1. Each module identifies records for files opened by *all* ranks.
2. Those records are **MPI-reduced** into a single aggregate record, written once.
3. Files opened by only one or some ranks keep their individual per-process records.

**Trade-off**

`DARSHAN_DISABLE_SHARED_REDUCTION=1` skips the reduction to keep full per-rank detail for shared files too, at the cost of much larger logs.

# File categories

Two ways to group the files of a job:

- **By access**: read-only, write-only, read/write (count, average and maximum size)
- **By sharing**:
  - **Unique**: opened by exactly one rank
  - **Shared**: opened by *every* rank
  - **Partially shared**: more than one rank, but not all

Look for: many small unique files $\rightarrow$ metadata load, shared files $\rightarrow$ contention between ranks.

# I/O cost

- Time spent in **read**, **write**, and **metadata** operations, per interface
- Averaged per process (summed over ranks / #processes)
- Compare against the job's total run time

Look for: metadata time dominating, I/O as a large share of run time.

# Operation counts

- Number of calls per operation type and interface, e.g. POSIX: read, write, open, stat, seek, mmap, fsync
- Shows which interface does the work, e.g. HDF5 on top of MPI-IO on top of POSIX

Look for: opens/stats far exceeding reads/writes. Keep the blind spots in mind.

# Access sizes

- Histogram of read/write sizes in 10 buckets, from 0–100 B up to $>$1 GiB
- The four most common access sizes per interface

Look for: many small accesses, especially far below the file system's block or stripe size.

# Access patterns

Compares each POSIX operation's offset with the previous one on the same file:

- **Consecutive**: starts exactly where the previous one ended
- **Sequential**: starts somewhere after the previous one (includes consecutive)
- Everything else: backwards jumps, i.e. random access

Look for: low sequential share $\rightarrow$ random access, poor for read-ahead and striping.

# Heat maps

- Rows: ranks, columns: time bins, color: bytes moved
- **Top edge**: sum across ranks per time slice $\rightarrow$ *when* was the job doing I/O?
- **Right edge**: sum across time per rank $\rightarrow$ was I/O balanced, or did a few ranks dominate?

Look for: bursty phases vs. continuous I/O, a single rank doing all the work.

# Questions to ask of a report

- Is time dominated by actual I/O, or by metadata operations (open/`stat`/`close`)?
- Are accesses large and contiguous, or small and scattered?
- Is I/O balanced across ranks, or do a few ranks dominate?
- Which interface (POSIX/MPI-IO/HDF5/…) is actually doing the work?

# Hands-on

Build Darshan, instrument HACC-IO, find a bottleneck

# Ground rules

- We go step by step. Every step ends with a **Check**: lines with `$` are what you type, the rest is what you should see. Wait until it works for you.
- Everything happens in `~/darshan-hands-on`.
- Commands are also in `hacc/MYREADME.md` in the hands-on repo, copy them from there, not from the PDF.
- Build on the login node, **run only via `sbatch`**.
- **Never** `export LD_PRELOAD` in your login shell: every `ls` and `git` would be instrumented.

# Step 1: hands-on repo and modules

bash

```
git clone <hands-on-repo-url> ~/darshan-hands-on
cd ~/darshan-hands-on
module purge
module load compiler/GCC devel/CMake mpi/OpenMPI lang/Python
```

Load the *same* modules for building and in every job script, otherwise the MPI libraries don't match.

**Check**

```
$ which mpicc
/apps/.../bin/mpicc
```

# Step 2: get Darshan 3.5.0

bash

```
cd ~/darshan-hands-on
git clone --branch 3.5.0 --depth 1 https://github.com/darshan-hpc/darshan.git
cd darshan
autoreconf -i
```

Pinning the version means everybody builds exactly the same code.

**Check**

```
$ ls configure
configure
```

# Step 3: configure and install

bash

```
./configure --prefix=$HOME/darshan-hands-on/darshan-install \
    --with-log-path=$HOME/darshan-hands-on/logs \
    --with-jobid-env=SLURM_JOB_ID CC=mpicc
make -j 8 install
```

- One build for both parts: darshan-runtime and darshan-util.
- Log path and job ID variable are required by `configure`.
- The summary at the end of `configure` lists the modules, look for `Lustre module support`.

**Check**

```
$ ls ~/darshan-hands-on/darshan-install/lib/libdarshan.so
/home/<user>/darshan-hands-on/darshan-install/lib/libdarshan.so
```

# Step 4: put the tools on your PATH

bash

```
export PATH=$HOME/darshan-hands-on/darshan-install/bin:$PATH
```

`libdarshan.so` is *not* preloaded here. The job scripts do that, only for the application.

**Check**

```
$ darshan-parser --version
3.5.0
```

# HACC-IO

- I/O kernel of the HACC cosmology code: checkpoint/restart of particle data
- Every rank writes *N* particles (9 arrays, 38 B per particle), reads them back, and verifies them
- Two builds of the same code:
  - `fpp`: one file per process
  - `shared`: one file for all ranks
- POSIX I/O only

# Step 5: build HACC-IO, twice

bash

```
cd ~/darshan-hands-on
git clone https://github.com/glennklockwood/hacc-io.git
cd hacc-io
make CXX=mpicxx clean fpp && cp hacc_io hacc_io.fpp
make CXX=mpicxx clean all && cp hacc_io hacc_io.shared
```

No Darshan linked in, we instrument at runtime.

**Check**

```
$ ls hacc_io.*
hacc_io.fpp  hacc_io.shared
```

# Step 6: the job script

~/darshan-hands-on/hacc/job.slurm (excerpt)

```
# TODO: fpp or shared
HACC_IO="$HANDSON/hacc-io/hacc_io.shared"
# HACC_IO="$HANDSON/hacc-io/hacc_io.fpp"

# TODO: change FS
OUTDIR="/lustre/project/.../${SLURM_JOB_ID}"
# OUTDIR="/localscratch/${SLURM_JOB_ID}/hacc"
```

Toggle by moving the #. The log name follows automatically, e.g. `hacc_io.shared_lustre_<jobid>.darshan`.

**vim**

`vim job.slurm`
`h j k l` move
`i` insert mode
`Esc` back to normal mode
`:wq` save and quit
`:q!` quit without saving

# Step 7: submit and watch

bash

```
cd ~/darshan-hands-on/hacc
sbatch job.slurm
squeue --me          # ST: PD = pending, R = running
cat hacc-io-darshan-<jobid>.out
```

- Pending for long? The `NODELIST(REASON)` column says why.
- No log? Check the `.err` file. A job killed by the time limit never reaches `MPI_Finalize`.

**Check**

```
$ ls ~/darshan-hands-on/logs
hacc_io.shared_lustre_<jobid>.darshan
```

# Step 8: four runs

| | Group A | Group B |
|---|---|---|
| 1. | `shared, lustre` | `shared, localscratch` |
| 2. | `fpp, lustre` | `fpp, localscratch` |
| 3. | `shared, localscratch` | `shared, lustre` |
| 4. | `fpp, localscratch` | `fpp, lustre` |

Each run: edit `job.slurm`, then `sbatch job.slurm`. Half of the room is group A, so not everyone hits the same file system at once.

**Check**

```
$ ls ~/darshan-hands-on/logs
hacc_io.fpp_localscratch_<jobid>.darshan
hacc_io.fpp_lustre_<jobid>.darshan
hacc_io.shared_localscratch_<jobid>.darshan
hacc_io.shared_lustre_<jobid>.darshan
```

# Step 9: read a full log

```bash
cd ~/darshan-hands-on/logs
darshan-parser hacc_io.shared_lustre_<jobid>.darshan | less
```

From top to bottom:

- **Job header**: exe, `nprocs`, `run time`
- **Log file regions**: which modules have data
- **Mounted file systems**: mount point and type
- **Records**, one counter per line: module, rank (`-1` = shared, reduced), record ID, counter, value, file name, mount point, file system

Find `POSIX_OPENS`, `POSIX_BYTES_WRITTEN`, `POSIX_F_WRITE_TIME`.

# Step 10: summaries

bash

```
darshan-parser --file hacc_io.shared_lustre_<jobid>.darshan
darshan-parser --perf hacc_io.shared_lustre_<jobid>.darshan
```

- `-file`: files, bytes, max. offset per category (`unique`, `shared`, …)
- `-perf`: bandwidth estimated from the slowest rank (`agg_perf_by_slowest`)
- Compare with your other logs: how do the `fpp` categories differ, which file system and pattern is faster?

**Check**

```
$ darshan-parser --file hacc_io.shared_lustre_<jobid>.darshan
# shared: 1 ...
```

# Step 11: where do the bytes come from?

```bash
darshan-parser --total hacc_io.fpp_lustre_<jobid>.darshan | less
```

- Expected: 32 ranks $\times$ 10 000 $\times$ 38 B $\approx$ 12 MB written
- Find `total_POSIX_BYTES_WRITTEN` and the access-size histogram `total_POSIX_SIZE_WRITE_*`

What does Darshan say, and why?

# Step 11: answer

## Darshan shows what the code really does

Every file starts with a fixed **24 MiB header**. With one file per process that is $32 \times 24\,\text{MiB} = 768\,\text{MiB}$, written and read back again.

- The large accesses show up in the access-size histogram
- The `shared` run writes the header only once

# Step 12: Lustre layout

```bash
darshan-parser hacc_io.shared_lustre_<jobid>.darshan | less
```

- Search for `LUSTRE` (in `less`: type `/LUSTRE`)
- `LUSTRE_NUM_STRIPES`, `LUSTRE_COMP1_STRIPE_SIZE`: how the file is spread over the storage servers
- Only there if `configure` reported Lustre module support (Step 3)

# The youdo app

- Every rank produces 1 MiB per time step, 8 steps
- All blocks go into one file on Lustre
- It works, but is it doing the I/O well?

bash

```
cd ~/darshan-hands-on/youdo
make
sbatch job.slurm
```

**Check**

```
$ cat youdo-darshan-<jobid>.out
...
32 ranks wrote 256 MiB in ... s
```

# Step 13: who does the I/O?

```bash
cd ~/darshan-hands-on/logs
darshan-parser --file youdo_<jobid>.darshan
darshan-parser youdo_<jobid>.darshan | less
```

- Is the file `unique` or `shared`?
- Look at the rank column of `POSIX_OPENS` and `POSIX_BYTES_WRITTEN`

32 ranks are running. How many of them touch the file?

# Step 13: answer

## Rank 0 does all the I/O

The file is *unique*: only rank 0 opens and writes it. The other 31 ranks send their data to rank 0 and wait.

- One rank's bandwidth instead of 32
- The waiting happens in `MPI_Gather`, which Darshan does not see
- A common pattern in real codes: "rank 0 writes the output"

# Step 14: fix it

Open `youdo.c` and find the two places marked `HANDS-ON`:

~/darshan-hands-on/youdo/youdo.c (excerpt)

```
// TODO 1: who opens the output file?
if (rank == 0)
    fd = open(argv[1], O_CREAT | O_WRONLY, 0644);
...
// TODO 2: who writes the data?
write_all_from_root(fd, block, step, rank, nprocs);
```

Both write functions are already in the file. Read them, change the two places, then `make` and `sbatch job.slurm`.

**Check**

```
$ cat youdo-darshan-<new jobid>.out
32 ranks wrote 256 MiB in ... s
```

# Step 15: compare before and after

bash

```
darshan-parser --file youdo_<new jobid>.darshan
darshan-parser youdo_<new jobid>.darshan | less
```

- The file is now `shared`: one record with rank `-1` (shared-file reduction)
- `POSIX_FASTEST_RANK` and `POSIX_SLOWEST_RANK` show how evenly the work is spread
- Compare the times in the two `.out` files

**Check**

```
$ darshan-parser --file youdo_<new jobid>.darshan
...
# shared: 1 ...
```

# Bonus: is the file complete?

- 32 ranks $\times$ 8 steps $\times$ 1 MiB: the file should be 256 MiB
- In the log after your fix, compare
  - `POSIX_BYTES_WRITTEN`: how much was written
  - `POSIX_MAX_BYTE_WRITTEN`: the largest offset written
- Do they fit together? If not, where did the data go?
- The intended layout is in the comment at the top of `youdo.c`

# Results

HTML reports with PyDarshan: an LLM inference run

# Step 16: PyDarshan on your own PC

```bash
python3 -m venv .darshan-venv
source .darshan-venv/bin/activate
pip install darshan
```

- Analysis needs no cluster: logs are portable, copy them anywhere
- The wheel works up to Python 3.13. On Python 3.14, build darshan-util yourself (`–disable-darshan-runtime`) and put `darshan-util/.libs` on `LD_LIBRARY_PATH`, see `analysis.md`

# The workload: vLLM with an LMCache KV cache

- LLM inference server vLLM, model Qwen3-8B (weights $\approx$ 16 GB), benchmark with 300 prompts sharing 200 prefixes
- LMCache stores the KV cache of prompt prefixes as files on local disk (`/tmp/lmcache_disk/`) and reuses them on a hit
- No MPI $\rightarrow$ one log per process. We look at `vllm_engine_core.darshan`: the EngineCore process loads the weights, runs the model, and does the KV I/O
- Darshan limits raised in `darshan.conf` (`MODMEM`, `MAX_RECORDS`), DXT only for KV and weight files

# Step 17: generate the HTML report

bash

```
cd darshan-hands-on/vllm_lmcache_inference
python -m darshan summary darshan/vllm_engine_core.darshan --enable_dxt_heatmap
```

Open the HTML file in your browser.

**Check**

```
$ ls *_report.html
vllm_engine_core_report.html
```

# Step 17: questions to the EngineCore report

1. Model load vs. serving: compare the weight reads (`.safetensors`) with the KV-cache I/O
2. How many KV-cache files are there, and how big is each? Are they written once and read many times?
3. Access sizes: large requests or small ones?
4. Metadata cost: compare metadata time with read/write time
5. Heat map: can you see startup, weight load, and serving?
6. Why did `darshan.conf` have to raise `MAX_RECORDS` and `MODMEM`?

# Answer: file count summary

1 process, 766 s, 17 795 files, 39 GiB read, 224 GiB written

| Category | Files | Avg. size | What it is |
|---|---|---|---|
| read-only | 11 046 | 1.47 MiB (max 3.72 GiB) | Python imports, model weight shards |
| write-only | 5 845 | 36 MiB | KV chunks written, never read back |
| read/write | 651 | 36 MiB | KV chunks written and reused |

## The application's chunk size, from the log alone

Most common access size: 37 748 736 B (6 871 $\times$) $=$ 256 tokens $\times$ 147 456 B per token $=$ 36 MiB, one KV chunk.

# Answer: data access per mount point

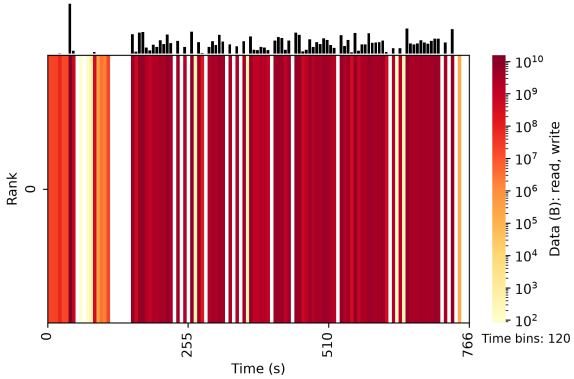
| | | |
|---|---|---|
| `/tmp` | 224 GiB written, 23 GiB read | LMCache disk tier, 6 492 KV files |
| `/home` | 15.3 GiB read, 206 files | Qwen3-8B weights and config |
| `/opt` | 677 MiB read, 10 847 files | Python venv: imports, `.so`, `.pyc` |

- $651 \times 36$ MiB $\approx 22.9$ GiB: every reused chunk is read exactly once
- Only $\approx 10\,\%$ of the written chunks are ever read back

## Discussion

224 GiB written for 23 GiB of cache hits: about 10:1 write amplification. Is the cache worth it for this workload?

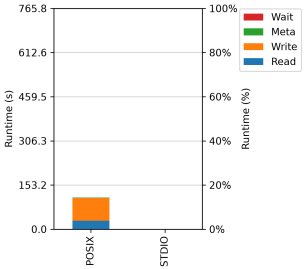
# Answer: heat map

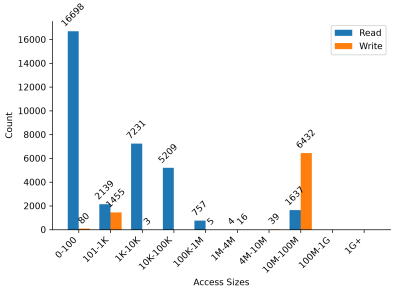



- **0–75 s**: Python imports, spike at ≈40 s is the weight load (15 GiB)
- **75–150 s**: almost no I/O, CUDA graph capture and JIT kernel builds
- **150–740 s**: benchmark, continuous KV writes and reads, $10^{9}$–$10^{10}$ B per bin

HEATMAP module, 120 bins of 6.4 s

# Answer: I/O cost and access sizes




- 105 of 766 s in POSIX I/O, 75 s write, 30 s read, metadata hardly visible
- Time *inside* system calls, maybe in background threads: 14 % I/O $\neq$ 14 % slower
- Two worlds: small Python reads, 36 MiB KV writes

Access pattern: only 1 537 of 8 030 writes are sequential. Not bad I/O: every KV file is one write at offset 0, and the first access to a file never counts as sequential.

## Metadata cost: Darshan can’t really answer this

Failed lookups of the Python import path search, `stat()` on new glibc, `readdir`, `mkdir`, `unlink`: not recorded (see *Blind spots*). “Hardly visible” is a lower bound. Every metadata call with a timestamp: **DFTracer, tomorrow**.

# Bonus: read and write heat maps separately

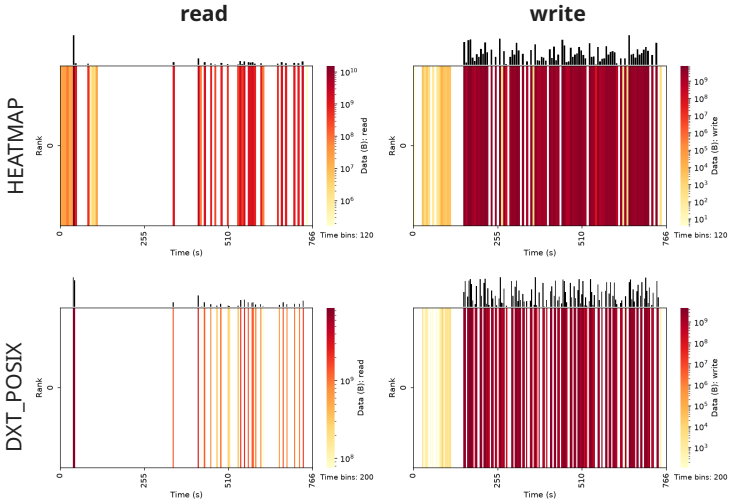
rw_heatmaps.py, usage: python rw_heatmaps.py <log>.darshan

```
import sys
import darshan
from darshan.experimental.plots.plot_dxt_heatmap import plot_heatmap

r = darshan.DarshanReport(sys.argv[1], read_all=True)
for op in ("read", "write"):
    # HEATMAP module: all POSIX I/O, coarse bins
    plot_heatmap(r, mod="HEATMAP", submodule="POSIX", ops=[op]).savefig(f"
    heatmap_posix_{op}.png", dpi=150)
    # DXT trace: only files passing the DXT filter, finer bins
    plot_heatmap(r, mod="DXT_POSIX", ops=[op]).savefig(f"heatmap_dxt_{op}.png",
     dpi=150)
```

Where are the weight reads, where are the cache hits?

# Bonus: answer




- **Reads**: weight load at ≈40 s, cache hits only from ≈340 s on
- **Writes**: KV chunks from 150 s, all through the run
- Split by operation, the DXT colour scale is sane again

# What we learned with Darshan

- **HACC-IO**: the code writes much more than expected, a 24 MiB header per file
- **youdo**: rank 0 did all the I/O, and the fix had an offset bug that the program itself never reported
- **vLLM**: the application's chunk size and its write amplification can be read from the log
- Without MPI: one log per process, raise the record limits for Python
- Darshan shows *what* happens, not *why*: always read the counters in context

# Sources & thanks

Content in this deck is original, but researched directly from Darshan's official documentation and inspired by / cross-checked against prior training material:

- Darshan project docs (darshan-runtime, darshan-util, darshan-modularization, PyDarshan usage docs, ChangeLog), Argonne National Laboratory
- Darshan project blog, March 2025 post on non-MPI and AI/ML instrumentation (Argonne National Laboratory)
- Jannis Klinkenberg & Radita Tapaning Hesti Liem (RWTH Aachen IT Center)
- Phil Carns (Argonne National Laboratory, Darshan lead developer, ATPESC)
- Shane Snyder, Kevin Harms, Rob Latham, Rob Ross (Argonne National Laboratory, ECP)
- `radix-io/hands-on` (exercise pattern reference only)

See `docs/credits-sources.md` for full details.

# Runtime configuration

Set as environment variables, or as keys in a config file given by `DARSHAN_CONFIG_PATH` (site installs may already set one).

| | |
|---|---|
| `DARSHAN_LOGFILE` | full path of the log to write |
| `DARSHAN_LOGPATH` | log directory (needs `darshan-mk-log-dirs` layout) |
| `DARSHAN_ENABLE_NONMPI=1` | instrument non-MPI programs |
| `DARSHAN_MOD_ENABLE` / `_DISABLE` | switch modules, e.g. `DXT_POSIX,DXT_MPIIO` |
| `DXT_ENABLE_IO_TRACE=1` | deprecated way to enable DXT |
| `DARSHAN_MODMEM` | memory for *all* modules together, in MiB (default 4) |
| `DARSHAN_DISABLE=1` | turn off all instrumentation |

# Incomplete logs

- Darshan's memory is fixed at startup: 1024 records per module, 4 MiB shared by all modules (`MODMEM`), 1 MiB for file names (`NAMEMEM`).
- When full, further files are not recorded and the module is marked *incomplete* (red warning in the report).
- Python and AI/ML jobs hit this quickly: every imported module and shared library is a file.
- Fix: raise `DARSHAN_MODMEM`, or `MAX_RECORDS` in a config file.

# Other analysis tools

| | |
|---|---|
| `darshan-parser` | all counters as text, `–perf`, `–file` for summaries |
| `darshan job_stats / file_stats` | rank jobs or files across many logs (PyDarshan) |
| `darshan-diff` | compare two logs |
| `darshan-convert` | upgrade, extract, or anonymize a log |